

CONFIG DRIFT DETECTOR

Project Documentation

1. Abstract

Configuration Drift Detector is an AI-powered system designed to identify and analyze differences between a baseline (golden) configuration and the current system configuration. The application helps organizations detect unauthorized changes, assess their impact, classify severity levels, and generate detailed reports. The system also provides AI-driven analysis to help understand the consequences of configuration drift.

2. Introduction

Configuration drift occurs when the actual configuration of a system deviates from its intended or baseline configuration over time. Such deviations can lead to security vulnerabilities, compliance violations, performance degradation, and operational failures.

The Config Drift Detector automates the process of identifying these deviations and provides actionable insights through AI-generated analysis and reporting.

3. Problem Statement

Organizations manage numerous servers, cloud resources, and applications. Manual tracking of configuration changes is difficult and error-prone. Undetected configuration drift can result in:

- Security breaches
- Compliance failures
- System instability
- Increased maintenance costs

A solution is required to automatically detect, analyze, and report configuration changes.

4. Objectives

- Detect configuration drift automatically.
 - Compare current and baseline configurations.
 - Classify drift severity.
 - Provide AI-powered impact analysis.
 - Generate detailed PDF reports.
 - Maintain drift history for auditing purposes.
-

5. Scope

The project can be used in:

- Cloud Infrastructure Monitoring
 - DevOps Environments
 - IT Compliance Audits
 - Network Configuration Management
 - Enterprise Security Operations
-

6. System Architecture

Modules

User Interface

Allows users to upload configuration files and view results.

Drift Detection Engine

Compares baseline and current configurations using DeepDiff.

AI Analysis Module

Uses LLM models to evaluate the impact of detected drifts.

Database Module

Stores drift records and historical information.

Report Generation Module

Creates PDF reports containing drift details and analysis results.

7. Technology Stack

Component	Technology
Programming Language	Python
Frontend	Streamlit
Database	SQLite
AI Model	Groq LLM
Diff Engine	DeepDiff
PDF Generation	ReportLab

8. Project Structure

Project/

```
|
|— database/
|— output/
|— static/
|— tickets/
|
|— app.py
|— drift_detector.py
|— ai_analysis.py
|— database.py
|— report_generator.py
|— requirements.txt
```

9. Workflow

1. User uploads Golden Configuration.
 2. User uploads Current Configuration.
 3. Drift Detection Engine compares both files.
 4. Differences are identified.
 5. Severity levels are assigned.
 6. AI Impact Analysis is generated.
 7. Results are stored in the database.
 8. PDF report is generated.
-

10. Functional Requirements

FR1

System shall accept configuration files.

FR2

System shall compare configurations.

FR3

System shall detect additions, deletions, and modifications.

FR4

System shall classify severity levels.

FR5

System shall generate PDF reports.

FR6

System shall maintain drift history.

11. Non-Functional Requirements

Performance

Drift detection should complete within seconds for standard configuration files.

Reliability

System should provide accurate comparison results.

Security

Sensitive configuration information should be protected.

Scalability

System should support multiple configuration comparisons.

12. Severity Classification

Low

Minor configuration changes with minimal impact.

Medium

Changes affecting performance or functionality.

High

Changes that may cause service disruption.

Critical

Changes introducing security risks or major failures.

13. Database Design

Drift_History Table

Field	Type
id	Integer
timestamp	DateTime
severity	Text

Field	Type
drift_count	Integer
report_path	Text

14. Test Cases

Test Case 1

Objective: Verify identical configurations.

Input: Same baseline and current file.

Expected Result: No drift detected.

Test Case 2

Objective: Detect modified values.

Input: Configuration with changed parameters.

Expected Result: Drift detected.

Test Case 3

Objective: Detect missing entries.

Input: Configuration missing required keys.

Expected Result: Drift reported.

Test Case 4

Objective: Generate report.

Input: Detected drift.

Expected Result: PDF report created successfully.

15. Advantages

- Automated monitoring
- Faster issue detection
- Improved security
- Reduced manual effort

- Detailed reporting
 - AI-assisted analysis
-

16. Future Enhancements

- Multi-cloud support
 - Real-time monitoring
 - Dashboard analytics
 - CI/CD integration
 - Predictive drift detection
 - Role-based access control
-

17. Conclusion

The Config Drift Detector provides an efficient and intelligent solution for identifying configuration changes in IT environments. By combining automated drift detection, AI-driven analysis, reporting, and drift history management, the system enhances operational reliability, security, and compliance management.